

pyfs-matrix cheatsheet

Typical flow: inventory (new mesh) → plan (mandatory, no solver) → run (--resume to add points) → collect (cluster jobs) → post | Safe by default: storage, sync, restore, rebuild and mark-failed only PREVIEW until --apply. --workspace defaults to the current folder; --runs NAME picks another manifest than runs.json; a matrix name is found in the workspace root or inputs/matrices/.

1. PREPARE THE WORKSPACE AND THE MATRIX

inventory <geometry> write <stem>.boundaries.toml beside a saved simulation (.fsm, from its mesh block) or an OBJ (from its groups); names the solver actions saved in a .fsm; no executable needed

- overwrite rewrite an existing .fsm sidecar (an OBJ sidecar is never rewritten)
- clean remove the saved solver actions (26.120 in meters: also reset to a fresh import, meshes and BCs kept); copy kept as <file>.bak-<stamp>

upgrade <matrix> convert a matrix written under an older layout

- in-place rewrite the file (default: print to stdout)
- inputs DIR also move inputs/groups/e*.toml into pproc [groups]; needs --in-place

rename rename a 0.20.x workspace to the 0.21.0 point names (folders, scripts, manifest, plan)

- workspace DIR --dry-run print what would move, change nothing

convert <matrix> emit the equivalent native campaign.toml

- fs-exe PATH required: FlightStream executable (never guessed)
- name NAME campaign name (required here: no workspace)
- fs-version V build for rows with empty FS_BUILD, e.g. 26.124
- recipe CODE=MODULE:FUNCTION FS_SCRIPT code to recipe (repeatable)
- o, --output FILE write here instead of stdout

2. PLAN (MANDATORY BEFORE RUN, NO SOLVER)

plan <matrix> bind the matrix to inputs/, pre-flight every point, write the receipt run requires (pinned to the matrix digest; edit the matrix, plan again). Warns, never refuses, on saved actions in an unsteady geometry and on families a row lacks. Alias: inspect-setups

- workspace DIR campaign root with inputs/
- name NAME campaign name (default: workspace folder name)
- fs-version V fallback build for empty FS_BUILD cells
- fs-exe PATH executable override; mandatory for MANUAL rows (else FS_BUILD via inputs/executables.toml)
- workflow CODE=NAME FS_SCRIPT code to run type (repeatable)
- recipe CODE=MODULE:FUNCTION LEGACY recipe (repeatable)
- point-name TPL naming template; default P<sim>-<point name>; placeholders {point} {alpha} {beta} {mach} {advance_ratio} {sim} {campaign}
- ignore-missing-families [true|false] default true: a family the mesh lacks is skipped; false refuses the point
- no-ignore-missing-families same as false
- cost table mesh size, TEs, farfield layers, viscous coupling, steady/unsteady, iterations, CPUs and an EXPECTED wall time fitted on this workspace's runs
- inflow-fft for qsteady_rotor wheel points in a custom inflow: blade-frame harmonics, k_eff, n_max, suggested PASSAGE_POSITIONS (warns if too few)
- update-ids (--updateIDs) REWRITE this matrix so no POL repeats one used elsewhere; written before the plan runs
- accept-unregistered-build allow an installed build other than the registered one (your responsibility; recorded)
- setup-guidelines write inputs/setups/SETUP_GUIDELINES.md
- setup-standards write commented s9XX setup standards
- verbose full warnings (all detail also in logs/activity.log)

3. RUN

run <matrix> run every active point and write the sweep table. Needs a current plan. Linux with a submission profile submits to the cluster; Windows runs locally

- resume skip points already recorded (a grown matrix runs only new points); without it a recorded point refuses
- force-rerun POINT REDO a recorded point (by point name, run_id or steady JOB id; repeatable). Old records and outputs are archived, nothing deleted. Not with --resume
- force-rerun-all REDO every recorded point (prints the license count first)
- sims SIM [SIM ...] with --force-rerun-all: only these simulations, space separated, e.g. --sims 2031 2032
- local run on THIS machine instead of submitting (Linux workstation or smoke test)
- progress-every N local unsteady progress every N steps (default 10; 0 = silent)
- sweep-csv FILE sweep table path (default post/<matrix>/campaign_sweep.csv)
- workflow CODE=NAME run type: steady, unsteady, unsteady_rotor, qsteady_rotor
- workspace, --name, --fs-version, --fs-exe, --recipe, --point-name, --ignore-missing-families, --no-ignore-missing-families, --accept-unregistered-build: as in plan
- pproc-warnings print grouped post-processing warnings
- verbose full warnings and one line per repeated item

4. COLLECT (CLUSTER OR INTERRUPTED RUNS)

collect for every SUBMITTED record, wait until its outputs are present and SETTLED, collect, assess, update the record, then post; HPC job_end_files present, no solver log: FAILED_EXECUTION

- watch keep sweeping until nothing is outstanding (default: one sweep)
- watch-interval S seconds between sweeps (default 60)
- interval S seconds between the two size/time observations (default 2)
- rounds N stop a watch after N sweeps
- sims IDS only these simulations, e.g. 2006,2007, and their post
- no-post collect only, do not rebuild products
- check-frozen REFUSE (not warn) averages touched by a frozen solve or an unread log block
- workspace, --runs NAME, --pproc-warnings, --verbose

Tip: nohup pyfs-matrix collect --watch > logs/collect.out 2>&1 &

5. POST-PROCESS (NO SOLVER, NO SEAT)

post [matrix] rebuild polars, sections and plot tables under post/<matrix>; no matrix = every matrix in the manifest. Old products are archived in post/<matrix>/archive/

- sims IDS only these simulations' products, in place (super files kept)
- runs NAME read another manifest (e.g. a rebuilt one)
- from-sims ignore runs.json, build records in memory from sims/ (runs made outside the package); products go to post/<matrix>@sims/
- steps-per-revolution N with --from-sims: for a LAST_REVS_AVG window
- additional-pproc reopen the final .fsm of rows with ADDITIONAL_PPROC (no solve, one launch per point) and extract that pproc; 26.124 only. With it: --fs-version, --fs-exe, --local, --recipe
- diagnostics print recorded post diagnostics as Markdown, change nothing
- strict exit 3 when a product was skipped by design (e.g. polar under sideslip)
- check-frozen refuse instead of warn (see collect)
- force-overwrite DESTROY products instead of archiving (asks; --yes answers it)
- workspace, --pproc-warnings, --verbose

6. SYNC WORKSPACES (E.G. CLUSTER TO WINDOWS)

sync {runs|post|fsm|all} run on the MAIN workspace; peers in inputs/sync-workspaces.toml; levels cumulative, runs < post < fsm < all. Nothing in main is deleted; inputs/ untouched but a declared matrix

- apply change files (default: preview)
- from NAME one peer (default: every non-main one)
- prefer-other on a record conflict take the peer's (default: main wins)
- overwrite on a file conflict archive main's copy and take the peer's
- restore with --apply: rebuild records of sims/ folders no record carries
- include-archives also bring archive folders (skipped by default)
- workspace, --runs NAME

7. RECOVER AND MARK RECORDS

restore {runs|storage|products|plan|additional} restore a file from archive/

- apply (default: preview) --stamp STAMP archive to restore from
- matrix STEM for products and plan, when several matrices have archives
- workspace

rebuild rebuild run records from the folders under sims/

- apply (default: preview) --out NAME write records to another manifest (compare before replacing)
- all-sims every simulation folder, recorded or not --sims 4001,2009 only these
- matrix FILE the matrix the simulations ran from
- build-alias BUILD=ALIAS scheduler name of a build (repeatable), e.g. 26.124=26.12
- inputs-from FOLDER another origin's inputs/ for inputs changed after the run
- workspace

mark-failed every record of the named simulations becomes FAILED_MARKED, its old status kept under marked; runs.json archived first. Not readable before 0.33.0

- sims 2006,2007 the simulations --reason TEXT why, recorded as given
- apply (default: preview) --workspace

8. DISK SPACE (EVERY CALL LOGGED IN storage_management.json)

space-in-use sizes by top folder, by sims/sim_*, by extension; changes nothing

- top N rows per grouping (default 15) --workspace

free-space <m-id> run storage recipe inputs/management/m<id>.toml: prune_step_exports, compact_sims (zip, restored automatically), delete_extensions, post_archives

- apply (default: preview) --runs NAME --workspace
- list every path each step touches, with its size and what happens to it

delete-sims <ids> delete simulations, their products and records, e.g. 4001,2009; a note row remains

- apply (default: preview) --runs NAME --workspace
- force also a simulation whose run is still SUBMITTED (refused without it)
- matrix-products {points-only|regenerate} for a matrix product that also holds the deleted points: leave it (marked stale) or rerun its post without them

Before a licensed run: plan with --cost; read the warnings; inventory --clean a geometry that carries saved actions. Exit codes of post: 0 done; 2 refused, nothing written; 3 with --strict, a product skipped by design. Trace: runs.json, plan.json, post/, logs/.

Every other command-line tool

The tools: pyfs-workspace builds the campaign tree and custom free-stream fields; pyfs-qa is the evidence tooling of the command database (probes and physics runs need a licensed machine); pyfs-fsi is the coupling executable the solver calls; pyfs-manual is a maintainer tool (the [manual] extra); the two python -m entries keep an optional workbook (the [excel] extra). Safe by default: the field operations preview until --apply; pyfs-manual writes only with --write.

pyfs-workspace

the campaign tree and the custom free-stream fields of inputs/freestreams/ init [root] create or complete the campaign tree: inputs/ with its library folders, sims/, post/, archive/; writes an inputs/executables.toml template when none exists. Default root: the current folder archive <root> <sim_id> zip one recorded simulation under archive/ and remove its folder sims/sim_<sim_id>; a simulation the manifest does not record is refused

migrate-geometries [root] move a flat inputs/geometries/ library into one folder per geometry

field build a custom free-stream file of inputs/freestreams/ from other fields. Every operation below previews until --apply, writes the file <STEM> (.txt STRUCTURED, .dat UNSTRUCTURED, the form decides) and its provenance record, and takes --out STEM (required), --workspace DIR, --apply, --overwrite (replace an existing <STEM> file and its record)

field mirror <field> mirror a field through the plane $x = 0$, $y = 0$ or $z = 0$

- plane {x,y,z} required: the coordinate whose sign changes
- out STEM --workspace --apply --overwrite

field move <field> translate a field so a source point lands on a target point

- source-point X Y Z required: the source point (a hub), m, global frame
- target-point X Y Z required: where it lands, m, global frame
- out STEM --workspace --apply --overwrite

field subtract <total> <other> total minus (other minus reference), point by point on one grid

- reference VX VY VZ required: the free stream OTHER was solved in, m/s (0 0 0 only where OTHER is already induced only)
- tolerance M two positions closer than this, per axis, are one point (default 1e-06)
- out STEM --workspace --apply --overwrite

field time-mean <files> the time mean of an unsteady run's per-step fields (files or glob patterns)

- last K average only the last K steps given (default: every step)
- fluctuation also write <stem>.fluctuation.csv, the per-probe standard deviation (two steps or more)
- fluctuation-only report the fluctuation only, no field (needs --last)
- vinf M_S free-stream speed, m/s, the fluctuation summary is also stated against
- tolerance M as in subtract
- out STEM --workspace --apply --overwrite

field fill-interior <field> fill the probes inside the body from the ray outside it

- r-body M body radius about the x axis, m (default 0.38)
- out STEM --workspace --apply --overwrite

pyfs-fsi

the coupling executable the solver's aerelastic toolbox calls once per time step. With no arguments it runs one coupling step in the current folder: coupled when config.json is there, the dummy (zero displacements) otherwise

init-dummy write the dummy configuration into a run folder

- node-count N required --dir DIR run folder (default: current)

step execute one coupling step explicitly

- dir DIR run folder (default: current)

pyfs-qa

evidence tooling of the command database: probes run the solver locally on a licensed machine

probe run command-validity probes and write the compat report pair

- fs-version V required: canonical build, e.g. 26.120
- fs-exe PATH required: the executable (never guessed)
- commands LIST comma-separated subset (default: every command with a probe spec)
- identity-only judge no command: capture the solver's identity banner, the way a new build gets its number into a committed report
- fsm FILE local saved simulation for preludes that need one
- workroot DIR scratch root (default runs/probes)
- timeout S per-probe limit, seconds (default 120)
- report-dir DIR (default reports/compat) --label TEXT report stem suffix

apply-compat <report> promote database statuses from a committed compat report

- root DIR repository root the report is cited against (default .)

physics run the physics matrix of a workspace, reduce its records, write the physics report

- workspace DIR workspace with the physics matrix at its root (default: current folder)
- matrix FILE (default matriz_physics.fs) --name NAME campaign name
- resume skip points already recorded
- report-dir DIR (default reports/physics) --label TEXT

drift run the physics matrix on two builds, each under its own registry, and diff the reductions

- fs-versions A,B required: baseline first
- fs-exe VERSION=PATH required, repeatable: one executable per version
- workroot DIR where the two sides are made (default runs/drift); a side that exists is refused
- workspace, --matrix, --name, --report-dir, --label: as in physics
- cases print the Tier 3 test matrix, one line per case id; nothing runs
- include-smi also the local-geometry class, never listed implicitly

cost <campaign> wall-time cost per simulation and point from runs.json, one column per build; runs nothing

- compare BASELINE,CANDIDATE two build columns, baseline first

update-reference <case> update or seed one physics reference from a committed physics report

- from-report FILE required --reason TEXT required, recorded in the reference

pyflightstream 0.33.0 every subcommand and option <tool> <cmd> -h (--help) gives the full text

pyfs-manual

maintainer tool, the [manual] extra: compares a vendor manual with the command database. The manual path is passed explicitly and nothing of it is copied

coverage what the manual documents and the database does not

- manual PDF --source ID --chapter-pages FIRST-LAST all required
- index-pages FIRST-LAST the Script Index (optional)

draft render database entries for the absent commands, for review

- manual, --source, --chapter-pages, --index-pages: as in coverage
- fs-version CANONICAL required, repeatable
- only TEXT filter on the command name --out FILE destination
- write actually write (default: print, touch nothing)

sweep read every edition; report what no entry covers

- editions MANIFEST required: YAML, one row per build
- by-section group by section --fail-if-absent exit 1 when a documented command cannot be emitted

surface the command surface of each edition and what changed between builds

- editions MANIFEST required, in release order
- names list the names, not only counts --markdown render as Markdown

citations re-read the database's page citations against the manuals

- editions MANIFEST required

register write documented rows for a new edition where the documentation did not change

- editions MANIFEST required --fs-version CANONICAL required
- commands-dir DIR chapter files to edit (default: the installed package)
- write actually write (default: print)

python -m pyflightstream.workspace.excel

the optional macro-free workbook, the [excel] extra; save and close it first create <output> create a new workbook

- workspace DIR required

preview <workbook> inspect changes without applying them (a batch JSON and its HTML)

- workspace DIR required --batch FILE required
- direction {read,write} required: read imports matrices, write exports
- matrix FILE select a subset (repeatable)

apply <batch> apply one fresh reviewed batch (single use)

cancel <batch> cancel a preview without changing inputs

check <workbook> validate its Runs and Dictionary

python -m pyflightstream.workspace.excel_bridge

the text bridge the workbook's own controls call: <action> is preview, apply or check; --request FILE, --response FILE and --batch FILE are all required

Copyright (c) 2026 Geovana Neves, CC BY 4.0.
https://github.com/nevesgeovana/pyflightstream